

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Đoàn Minh Quân

**DBARCHITECT: CÔNG CỤ TỰ ĐỘNG HÓA THIẾT
KẾ CƠ SỞ DỮ LIỆU VÀ SINH MÃ NGUỒN DỰA TRÊN
MÔ TẢ PHẦN MỀM**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



ĐOÀN MINH QUÂN

**DBARCHITECT: CÔNG CỤ TỰ ĐỘNG HÓA THIẾT
KẾ CƠ SỞ DỮ LIỆU VÀ SINH MÃ NGUỒN DỰA TRÊN
MÔ TẢ PHẦN MỀM**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: PGS.TS. Võ Đình Hiếu

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Doan Minh Quan

**DBARCHITECT: AUTOMATED TOOL FOR DATABASE DESIGN
AND SOURCE CODE GENERATION BASED ON SOFTWARE
DESCRIPTION**

BACHELOR'S THESIS

Major: Computer Science

Supervisor: Assoc. Prof. Dr. Vo Dinh Hieu

HANOI - 2026

Lời cảm ơn

Lời cam đoan

Tôi là ..., sinh viên lớp. Tôi xin cam đoan ...

Hà Nội, ngày tháng năm

Sinh viên

Nguyễn Văn A

Tóm tắt

Trong quy trình phát triển phần mềm hiện đại, việc thiết kế cấu trúc cơ sở dữ liệu đóng vai trò then chốt nhưng đòi hỏi chuyên môn cao và thường xuyên phải điều chỉnh lặp đi lặp lại. Mặc dù các công cụ hỗ trợ lập trình truyền thống đã tồn tại, việc chuyển đổi từ những yêu cầu kinh doanh mơ hồ sang một sơ đồ cơ sở dữ liệu chuẩn hóa vẫn là một thách thức lớn đối với nhiều nhà phát triển. Khóa luận này đề xuất xây dựng hệ thống DBArchitect, một giải pháp thông minh ứng dụng kiến trúc đa tác nhân nhằm tự động hóa quy trình thiết kế lược đồ cơ sở dữ liệu. Mục tiêu trọng tâm của dự án là tối ưu hóa việc trích xuất thực thể, chuẩn hóa quan hệ dữ liệu và tự động sinh mã nguồn khởi tạo, giúp giảm thiểu nỗ lực của con người trong giai đoạn đầu của vòng đời phát triển phần mềm. Hệ thống được thiết kế dựa trên quy trình làm việc hai giai đoạn được điều phối bởi LangGraph. Giai đoạn đầu sử dụng tác nhân Product Manager để làm rõ các yêu cầu không cấu trúc thông qua các câu hỏi tương tác. Giai đoạn hai sử dụng tác nhân Architect để tổng hợp thông tin thành lược đồ chuẩn dưới định dạng DBML. Kết quả thực nghiệm cho thấy DBArchitect có khả năng tạo ra các lược đồ cơ sở dữ liệu chuẩn hóa, hợp lệ về mặt cú pháp với sự can thiệp tối thiểu từ người dùng. Hệ thống chứng minh được hiệu quả trong việc xử lý các kịch bản thực tế phức tạp, giảm đáng kể thời gian thiết kế so với phương pháp thủ công truyền thống mà vẫn đảm bảo được tính toàn vẹn của dữ liệu và tuân thủ các quy ước chuẩn trong ngành công nghiệp phần mềm.

Từ khóa: multi-agent pattern, large language models, automated database design, code generation

Abstract

In modern software development, database structure design plays a crucial role but requires high expertise and frequent iterative adjustments. Although traditional programming tools exist, the transition from vague business requirements to a standardized database schema remains a significant challenge for many developers. This thesis proposes the development of DBArchitect, an intelligent solution applying a multi-agent architecture to automate the database schema design process. The central goal of the project is to optimize entity extraction, normalize data relationships, and automatically generate initial source code, minimizing human effort in the early stages of the software development lifecycle. The system is designed based on a two-stage workflow orchestrated by LangGraph. The first stage uses the Product Manager agent to clarify unstructured requirements through interactive queries. Phase two utilizes the Architect agent to synthesize information into a standardized schema in DBML format. Experimental results demonstrate that DBArchitect is capable of generating standardized, syntactically valid database schemas with minimal user intervention. The system proves effective in handling complex real-world scenarios, significantly reducing design time compared to traditional manual methods while ensuring data integrity and compliance with industry standards.

Keywords: multi-agent pattern, large language models, automated database design, code generation

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

Mục lục iv

Danh sách hình vẽ vi

Danh sách bảng vii

Danh mục các từ viết tắt viii

Chương 1 Đặt vấn đề 1

1.1 Bối cảnh và lý do chọn đề tài 1

1.2 Mô tả bài toán 1

1.3 Mục tiêu và phạm vi nghiên cứu 2

1.4 Cấu trúc khóa luận tốt nghiệp 2

Chương 2 Cơ sở lý thuyết 4

2.1 Giới thiệu về Generative AI và LLM 4

2.2 Ngôn ngữ mô hình hóa và biểu diễn dữ liệu 4

2.3 Các công nghệ sử dụng phía Client và Server 4

Chương 3 Phân tích và thiết kế hệ thống 5

3.1 Phân tích yêu cầu hệ thống 5

3.1.1 Yêu cầu chức năng 5

3.1.2 Yêu cầu phi chức năng 6

3.2 Kiến trúc tổng quan hệ thống 6

3.2.1 Tầng giao diện và tương tác người dùng	6
3.2.2 Tầng điều phối nghiệp vụ và quản lý dữ liệu	8
3.2.3 Tầng trí tuệ nhân tạo và xử lý đa tác nhân	11
3.2.4 Tầng lưu trữ dữ liệu	13
3.2.5 Luồng dữ liệu của hệ thống	13
3.3 Các biểu đồ mô tả chi tiết thiết kế hệ thống	15
3.4 Thiết kế cơ sở dữ liệu	15
3.4.1 Mô tả thực thể dữ liệu	15
3.4.2 Từ điển dữ liệu	16
Chương 4 Thực nghiệm và đánh giá	17
4.1 Môi trường triển khai và cài đặt hệ thống	17
4.1.1 Ngôn ngữ lập trình và Framework sử dụng	17
4.1.2 Cài đặt và triển khai hệ thống	18
4.1.3 Yêu cầu tiên quyết	18
4.1.4 Cấu hình cơ sở dữ liệu	18
4.1.5 Chạy Python Agent Service	18
4.1.6 Chạy Java Backend	19
4.1.7 Chạy Frontend	19
4.2 Kịch bản kiểm thử	19
4.3 Kết quả kiểm thử	19
4.4 Phân tích và thảo luận kết quả	19
Kết luận	20
Tài liệu tham khảo	21

Danh sách hình vẽ

3.1	Kiến trúc tổng quan hệ thống	7
3.2	Sơ đồ các thành phần của Frontend	8
3.3	Sơ đồ kiến trúc của Java Backend	10
3.4	Sơ đồ kiến trúc của dịch vụ AI	12
3.5	Sơ đồ kiến trúc cơ sở dữ liệu	14

Danh sách bảng

3.1	Đặc tả chi tiết thực thể Project	16
4.1	Tổng hợp đặc tả công nghệ và vai trò các thành phần trong hệ thống	17

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
BiLSTM	Bidirectional Long Short Term Memory	Mô hình bộ nhớ dài hạn ngắn hạn hai chiều
BO	Buffer Overflow	Lỗi tràn bộ đệm

Chương 1

Đặt vấn đề

1.1 Bối cảnh và lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số, dữ liệu đã trở thành tài sản chiến lược của mọi doanh nghiệp, khiến việc thiết kế một cơ sở dữ liệu (CSDL) chuẩn hóa và tối ưu ngay từ giai đoạn đầu trở thành yếu tố then chốt để đảm bảo tính toàn vẹn cũng như khả năng mở rộng của hệ thống. Tuy nhiên, quy trình này hiện đang đối mặt với nhiều thách thức lớn, trước hết là rào cản về chuyên môn khi việc thiết kế đòi hỏi kiến thức sâu rộng về chuẩn hóa, quan hệ thực thể và tối ưu hóa truy vấn – những kỹ năng mà các nhà phát triển mới vào nghề thường lúng túng khi phải chuyển đổi yêu cầu kinh doanh phức tạp thành lược đồ kỹ thuật. Bên cạnh đó, sự thiếu hụt các công cụ thông minh cũng là một trở ngại đáng kể, bởi trong khi các công cụ truyền thống như MySQL Workbench hay Draw.io chỉ dừng lại ở mức hỗ trợ vẽ thủ công, thì các mô hình ngôn ngữ lớn đơn lẻ lại thường gặp hiện tượng "ảo giác" hoặc sinh ra kết quả thiếu nhất quán đối với các yêu cầu mơ hồ. Chính những hạn chế này đã dẫn đến tính chất lặp lại và tốn kém thời gian trong quá trình trao đổi giữa bộ phận nghiệp vụ và kỹ thuật, gây ra nhiều vòng lặp không đáng có và trực tiếp làm chậm trễ tiến độ của toàn bộ dự án.

Xuất phát từ thực tiễn đó, đề tài "DBArchitect: Công cụ tự động hóa thiết kế cơ sở dữ liệu và sinh mã nguồn dựa trên mô tả phần mềm" được lựa chọn nhằm kết hợp sức mạnh của Generative AI và quy trình kỹ thuật phần mềm chuẩn mực, giúp tự động hóa và nâng cao chất lượng thiết kế cơ sở dữ liệu.

1.2 Mô tả bài toán

Bài toán mà KLTN giải quyết là tự động hóa quy trình từ phân tích yêu cầu đến sinh lược đồ CSDL và mã nguồn khởi tạo thông qua sự phối hợp của các tác nhân AI chuyên biệt.

Đầu vào (Input): Các yêu cầu nghiệp vụ dưới dạng ngôn ngữ tự nhiên (tiếng

Anh/tiếng Việt), có thể là các đoạn mô tả ngắn hoặc các tài liệu yêu cầu thô, chưa cấu trúc.

Đầu ra (Output):

Lược đồ CSDL dưới định dạng chuẩn DBML.

Sơ đồ thực thể liên kết (ERD) trực quan, có thể tương tác.

Mã nguồn boilerplate (Spring Boot Entities) để triển khai ngay lập tức.

Phạm vi: Tập trung vào các hệ quản trị CSDL quan hệ (RDBMS). Hệ thống hỗ trợ từ giai đoạn khơi gợi yêu cầu đến giai đoạn đặc tả lược đồ mức logic.

Giả định: Người dùng có hiểu biết cơ bản về các thực thể cần quản lý; hệ thống giả định các yêu cầu nhập vào không vi phạm các chính sách bảo mật thông tin nghiêm ngặt.

1.3 Mục tiêu và phạm vi nghiên cứu

Mục tiêu nghiên cứu:

Nghiên cứu cơ chế điều phối đa tác nhân (Multi-agent) sử dụng LangGraph để giảm thiểu sai số của AI.

Xây dựng tác nhân Product Manager có khả năng đặt câu hỏi thông minh để làm rõ các điểm mơ hồ trong yêu cầu của người dùng.

Xây dựng tác nhân Architect có khả năng tư duy logic để thiết kế lược đồ chuẩn hóa và tự sửa lỗi cú pháp.

Phát triển một nền tảng Web hoàn chỉnh cho phép người dùng giám sát và điều chỉnh kết quả thiết kế. Phạm vi nghiên cứu Công nghệ: Gemini 2.5 Flash, LangGraph, Spring Boot, React Flow.

Giới hạn: Tập trung vào thiết kế lược đồ mức logic (Logical Schema), chưa đi sâu vào tối ưu hóa mức vật lý (Physical Tuning) như Indexing hay Partitioning chuyên sâu.

1.4 Cấu trúc khóa luận tốt nghiệp

Khóa luận được chia thành 4 chương chính:

Chương 1: Đặt vấn đề – Giới thiệu bối cảnh, lý do chọn đề tài, mục tiêu và phạm vi nghiên cứu.

Chương 2: Nền tảng và công nghệ sử dụng – Trình bày kiến thức cơ sở về LLM, Multi-agent, DBML và các thư viện hỗ trợ.

Chương 3: Phân tích và thiết kế hệ thống – Chi tiết về kiến trúc đa tác nhân, luồng xử lý của LangGraph và thiết kế các thành phần hệ thống.

Chương 4: Cài đặt, kiểm thử và triển khai hệ thống – Mô tả quá trình hiện thực hóa phần mềm, các kịch bản kiểm thử và phân tích kết quả đạt được.

Chương 2

Cơ sở lý thuyết

2.1 Giới thiệu về Generative AI và LLM

2.2 Ngôn ngữ mô hình hóa và biểu diễn dữ liệu

2.3 Các công nghệ sử dụng phía Client và Server

Chương 3

Phân tích và thiết kế hệ thống

3.1 Phân tích yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

Nhóm chức năng xử lý ngôn ngữ tự nhiên

Tiếp nhận yêu cầu: Hệ thống cho phép người dùng nhập mô tả bài toán bằng ngôn ngữ tự nhiên, hỗ trợ Tiếng Anh và Tiếng Việt.

Làm rõ yêu cầu: Tác nhân Product Manager phải có khả năng phân tích sự thiếu sót trong mô tả ban đầu và đưa ra các câu hỏi gợi ý.

Thiết kế lược đồ tự động: Tác nhân Architect trích xuất thực thể, thuộc tính và xác định các mối quan hệ (1-1, 1-n, n-n) từ các yêu cầu đã chốt.

Tối ưu hóa và sửa lỗi: AI tự động kiểm tra tính hợp lệ của cú pháp DBML và thực hiện các vòng lặp sửa lỗi nếu kết quả sinh ra không đúng định dạng.

Nhóm chức năng quản lý sơ đồ

Trực quan hóa: Hiển thị sơ đồ ERD dưới dạng các khối (nodes) và đường liên kết (edges) cho phép người dùng quan sát tổng thể.

Tương tác trực tiếp: Người dùng có thể kéo thả, thêm/xóa bảng hoặc thay đổi kiểu dữ liệu của các cột ngay trên giao diện.

Đồng bộ hóa: Mọi thay đổi trên giao diện phải được cập nhật ngược lại vào mã nguồn DBML.

Nhóm chức năng sinh mã và xuất dữ liệu

Sinh mã nguồn: Tự động tạo mã nguồn Java Spring Boot dựa trên thiết kế cơ sở dữ liệu cuối cùng.

Xuất định dạng: Cho phép xuất kết quả ra các định dạng như SQL script, DBML file hoặc hình ảnh sơ đồ.

3.1.2 Yêu cầu phi chức năng

Độ chính xác: Lược đồ CSDL sinh ra phải tuân thủ ít nhất dạng chuẩn 3 (3NF) trong hầu hết các trường hợp phổ biến.

Thời gian phản hồi: Do gọi API từ LLM thường mất thời gian, hệ thống cần có cơ chế thông báo trạng thái (loading) hoặc xử lý bất đồng bộ để người dùng không cảm thấy bị treo máy.

Tính nhất quán: Cùng một bộ yêu cầu nên cho ra các kết quả có cấu trúc tương đồng, tránh việc AI sinh ra mỗi lần một kiểu khác hẳn nhau.

Tính khả dụng: Giao diện thiết kế database phải trực quan, dễ sử dụng cho cả những người không chuyên về công cụ vẽ.

Tính mở rộng: Kiến trúc Agent cần linh hoạt để sau này có thể thêm các Agent mới (ví dụ: Agent tối ưu hóa hiệu năng, Agent bảo mật).

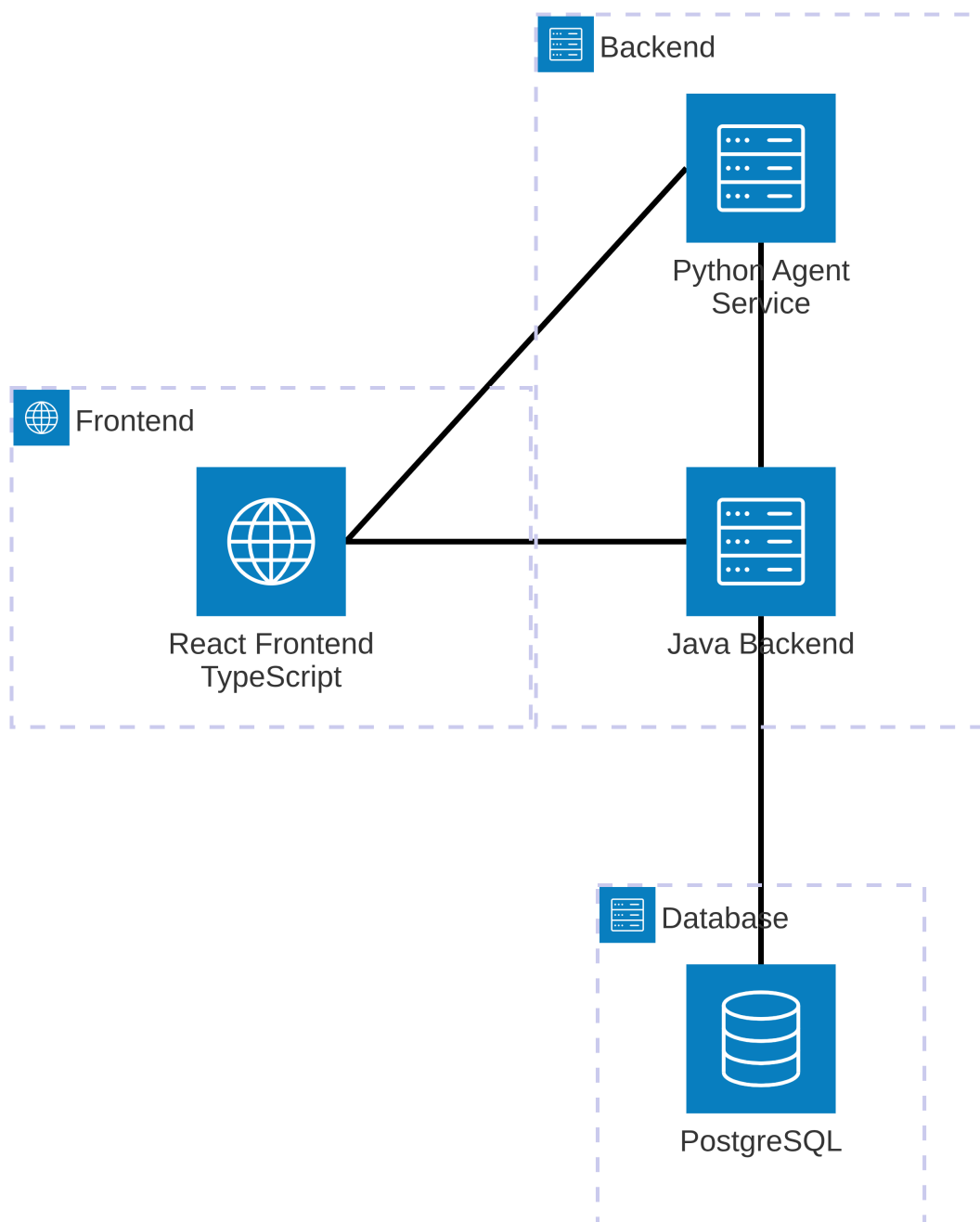
3.2 Kiến trúc tổng quan hệ thống

Hình 3.1 biểu diễn kiến trúc tổng quan của hệ thống.

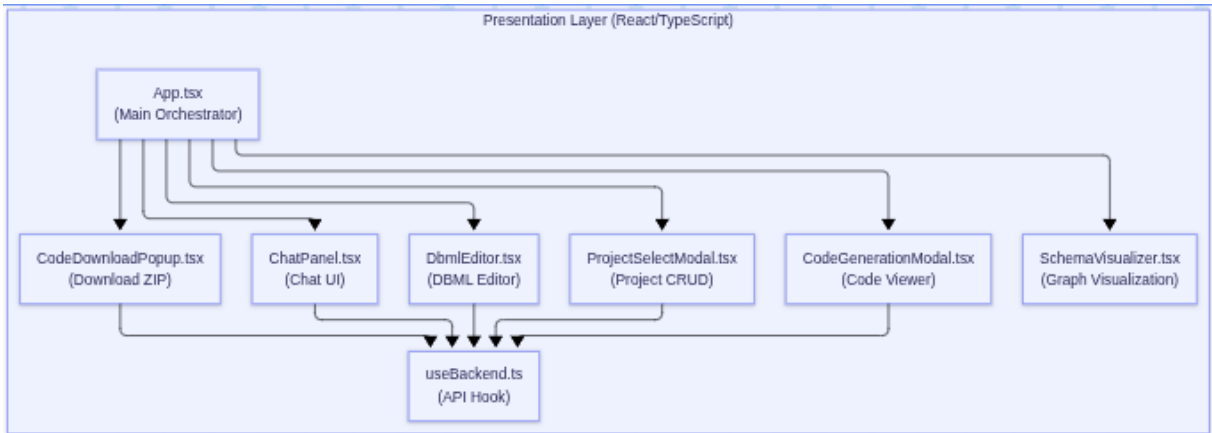
3.2.1 Tầng giao diện và tương tác người dùng

Giao diện người dùng của DBArchitect được triển khai dưới dạng ứng dụng React một trang viết bằng TypeScript, với logic điều phối chính được tập trung trong App.tsx. Về mặt kiến trúc, lớp này đóng vai trò là ranh giới trình bày và tương tác của hệ thống: nó thu thập yêu cầu của người dùng, hiển thị phản hồi hội thoại từ quy trình làm việc của AI, hiển thị lược đồ DBML đang phát triển và hiển thị các thao tác cấp dự án như tải dự án có sẵn, chỉnh sửa code DBML của dự án, tạo mới dự án và tải xuống mã nguồn của dự án. Thiết kế này duy trì trạng thái ở phía Client, nhưng trạng thái mà nó duy trì chỉ là trạng thái giao diện người dùng chứ không phải logic nghiệp vụ, điều này giúp giao diện người dùng phản hồi nhanh chóng trong khi vẫn giữ được sự tách biệt rõ ràng với các dịch vụ Backend.

Một đặc điểm đáng chú ý của lớp Frontend là quy trình làm việc hướng đối thoại. Người dùng bắt đầu bằng cách nhập tên dự án và mô tả bằng ngôn ngữ tự nhiên về hệ thống mà họ muốn mô hình hóa. Giao diện sau đó chuyển hướng yêu cầu thông qua bảng trò chuyện, nơi các câu hỏi làm rõ có thể được trả về khi hệ



Hình 3.1: Kiến trúc tổng quan hệ thống



Hình 3.2: Sơ đồ các thành phần của Frontend

thống xác định rằng các yêu cầu không đầy đủ và còn mơ hồ. Mô hình tương tác này được triển khai thông qua trạng thái và trình xử lý trong App.tsx và được hỗ trợ bởi các hàm tương tác với các dịch vụ Backend trong useBackend.ts. Kết quả là một vòng lặp tinh chỉnh yêu cầu lặp đi lặp lại thay vì chỉ gửi yêu cầu nghiệp vụ một lần. Cách thiết kế này phù hợp với các công việc thiết kế cơ sở dữ liệu dựa trên yêu cầu cầu phần mềm do các yêu cầu thường rất mơ hồ và không rõ ràng.

Về mặt chức năng, giao diện người dùng kết hợp một số thành phần con. ChatPanel quản lý đầu vào hội thoại và câu trả lời làm rõ. DbmlEditor cho phép chỉnh sửa trực tiếp DBML được tạo ra, cung cấp cho người dùng chức năng sửa chữa thủ công DBML sau khi AI tạo ra. SchemaVisualizer chuyển đổi DBML hiện tại thành chế độ xem lược đồ đồ họa, giúp người dùng kiểm tra cấu trúc bảng và các mối quan hệ một cách trực quan. Ngoài ra, các thành phần như ProjectSelectModal, CodeGenerationModal và CodeDownloadPopup hỗ trợ lựa chọn dự án và quản lý tạo mã. Cấu trúc mô-đun này cải thiện khả năng bảo trì vì mỗi trách nhiệm giao diện người dùng được đóng gói trong một thành phần chuyên dụng.

3.2.2 Tầng điều phối nghiệp vụ và quản lý dữ liệu

Từ góc độ hệ thống, giao diện người dùng cũng là trung tâm tích hợp cho dịch vụ Python Agent và Java Backend. Thay vì nhúng trực tiếp các logic nghiệp vụ vào giao diện người dùng, ứng dụng điều hướng toàn bộ quá trình giao tiếp với hai dịch vụ Backend cho useBackend.ts, cung cấp một giao diện API tập trung cho các chức năng như trò chuyện, tải dự án, cập nhật DBML, tạo mã, tạo bản xem

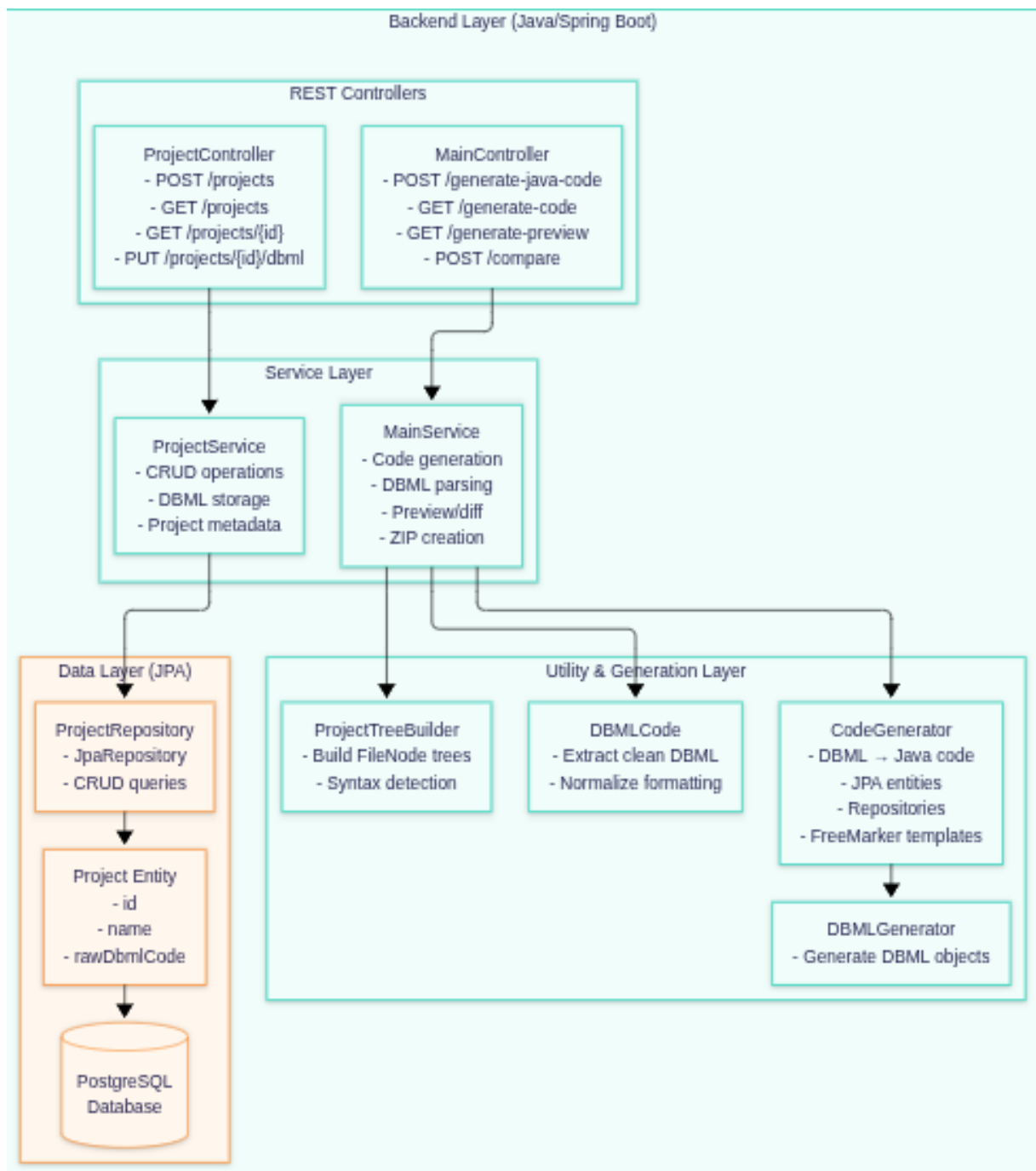
trước và tải xuống tệp ZIP. Cơ chế này giúp tách biệt các vấn đề về giao tiếp giữa Frontend và Backend khỏi các vấn đề về hiển thị trên Frontend và tạo ra một ranh giới dịch vụ rõ ràng cho ứng dụng. Do đó, giao diện người dùng hoạt động như một lớp điều phối ban đầu: nó quản lý ý định của người dùng, hiển thị các chuyển đổi trạng thái và chuyển tiếp yêu cầu đến dịch vụ Backend thích hợp.

Phần Java Backend bao gồm các lớp lưu trữ dữ liệu và tạo mã nguồn của hệ thống, được triển khai dưới dạng ứng dụng Spring Boot. Vai trò của Java Backend cung cấp các điểm cuối REST ổn định để quản lý dự án, chuyển đổi DBML thành mã nguồn Java được tạo sẵn và hỗ trợ các hoạt động tiếp theo như tạo bản xem trước và xuất ZIP. Bên cạnh đó, Java Backend nhận đầu vào có cấu trúc từ dịch vụ Python Agent và giao diện người dùng React, lưu trữ trạng thái dự án trong PostgreSQL.

Từ góc độ triển khai, phần backend tuân theo kiến trúc controller-service-repository thông thường của một ứng dụng Spring Boot. Các controller vẫn gọn nhẹ và sử dụng logic nghiệp vụ trong các lớp dịch vụ, điều này cải thiện sự phân tách các mối quan tâm và đơn giản hóa việc kiểm thử. File ProjectService.java chịu trách nhiệm cho các hoạt động CRUD trên thực thể Project, bao gồm tạo dự án, truy xuất, liệt kê và cập nhật DBML. MainService.java xử lý các quy trình tạo mã nguồn cấp cao hơn, bao gồm chuyển đổi DBML sang mã nguồn, xây dựng bản xem trước dự án, đóng gói ZIP và trả về file ZIP cho Frontend. Cấu trúc này giúp tách biệt các vấn đề về lưu trữ dữ liệu khỏi các vấn đề về tạo mã nguồn, điều này rất quan trọng vì phần backend phải hỗ trợ cả quản lý dự án vận hành và tổng hợp mã nguồn mà không được kết hợp chúng thành một dịch vụ nguyên khối, gây khó khăn trong việc phát triển về sau cũng như trong quá trình kiểm thử.

Lớp truy cập dữ liệu được triển khai thông qua Spring Data JPA, với file ProjectRepository.java cung cấp lớp trừu tượng hóa kho lưu trữ trên thực thể Project. Phần backend lưu trữ siêu dữ liệu dự án và code DBML thô trong PostgreSQL. Hibernate được sử dụng để ánh xạ mô hình miền sang lưu trữ quan hệ. Bên cạnh đó, các tiện ích chuẩn hóa DBML đảm bảo rằng các biểu diễn thô và biểu diễn sạch ¹ có thể được xử lý một cách nhất quán.

¹Hệ thống cần phải phân ra biểu diễn DBML thô và sạch vì biểu diễn thô có thể bảo toàn những gì người dùng hoặc agent viết, ví dụ như các dấu markdown, comment, dấu cách,... Hệ thống có cung cấp hàm để chuyển đổi giữa hai dạng này. Dạng code DBML đã được làm sạch sẽ



Hình 3.3: Sơ đồ kiến trúc của Java Backend

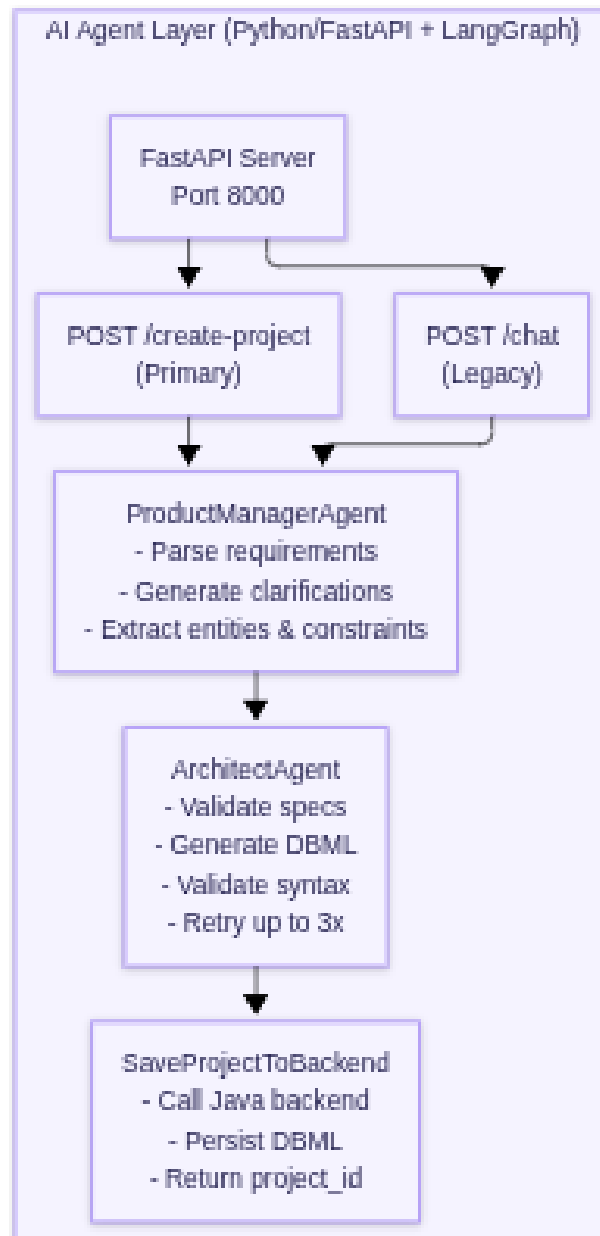
Một khía cạnh nổi bật của phần backend này là vai trò của nó trong việc tạo mã nguồn. Thay vì chỉ lưu trữ code DBML, nó chuyển đổi code DBML thành một khung sườn Spring Boot hoàn chỉnh, bao gồm các entity, repository, service và controller. Quy trình tạo mã dựa trên các lớp tiện ích như CodeGenerator.java, DBMLCode.java và ProjectTreeBuilder.java. Về mặt kiến trúc, điều này biến phần backend thành cả dịch vụ lưu trữ dữ liệu và công cụ chuyển đổi mã: nó lưu trữ code DBML, tạo ra mã nguồn từ đó và đóng gói kết quả để kiểm tra hoặc tải xuống.

3.2.3 Tầng trí tuệ nhân tạo và xử lý đa tác nhân

Dịch vụ Agent của hệ thống DBArchitect được triển khai bằng Python sử dụng FastAPI và LangGraph. Chức năng chính của nó là làm lớp trung gian giữa những yêu cầu không cấu trúc của người dùng và những thiết kế cơ sở dữ liệu có cấu trúc. Thay vì chấp nhận một đặc tả cứng nhắc duy nhất, dịch vụ hỗ trợ quy trình làm việc hội thoại lặp đi lặp lại, trong đó hệ thống có thể đặt câu hỏi làm rõ, tinh chỉnh các yêu cầu và chỉ sau đó mới tạo ra biểu diễn DBML cuối cùng. Hiện tại, dịch vụ Python Agent cung cấp endpoint /create-project để có thể vừa tương tác với Agent, vừa tự động tạo dự án mới và lưu cơ sở dữ liệu thông qua Java .

Về mặt kiến trúc, dịch vụ Python Agent được tổ chức thành một workflow được xây dựng xung quanh các tác nhân chuyên biệt. Tác nhân Product Manager chịu trách nhiệm diễn giải yêu cầu của người dùng, xác định xem yêu cầu này đã đủ rõ ràng chưa và tạo ra các câu hỏi làm rõ khi đầu vào còn mơ hồ và chưa đầy đủ. Sau đó, tác nhân Architect sẽ sử dụng các yêu cầu đã được làm rõ và đã được chuẩn hóa dưới dạng JSON rồi tạo ra DBML đã được kiểm thử bởi chính tác nhân Architect và công cụ validate_dbml. Sự phân tách này rất quan trọng vì nó phản ánh một quy trình thiết kế phần mềm thực tế: một giai đoạn tập trung vào việc thu thập yêu cầu, trong khi giai đoạn tiếp theo tập trung vào việc xây dựng và xác thực lược đồ.

Trong hệ thống tổng thể, dịch vụ Python Agent cũng là cầu nối giữa hội thoại và lưu trữ dữ liệu. Trong quy trình tạo dự án (create-project SOP), sau khi code DBML cũng như thiết kế cơ sở dữ liệu được hoàn thiện, dịch vụ sẽ gửi DBML được tạo ra đến Java Backend để dự án có thể được lưu trữ và sau đó có thể chỉnh sửa để giao tiếp giữa các dịch vụ và có thể hạn chế lỗi xảy ra.



Hình 3.4: Sơ đồ kiến trúc của dịch vụ AI

sửa hoặc tạo mã nguồn. Điều đó làm cho dịch vụ Python Agent không chỉ đơn thuần là một chatbot mà còn hoạt động như một dịch vụ điều phối chuyên biệt, chuyển đổi ý định của người dùng thành bản ghi dự án được lưu trữ trực tiếp xuống cơ sở dữ liệu. Trong cấu trúc kiến trúc của hệ thống, đây là thành phần mang lại cho DBArchitect khả năng tương tác linh hoạt cũng như khả năng nhận diện và xử lý các yêu cầu chưa rõ ràng.

3.2.4 Tầng lưu trữ dữ liệu

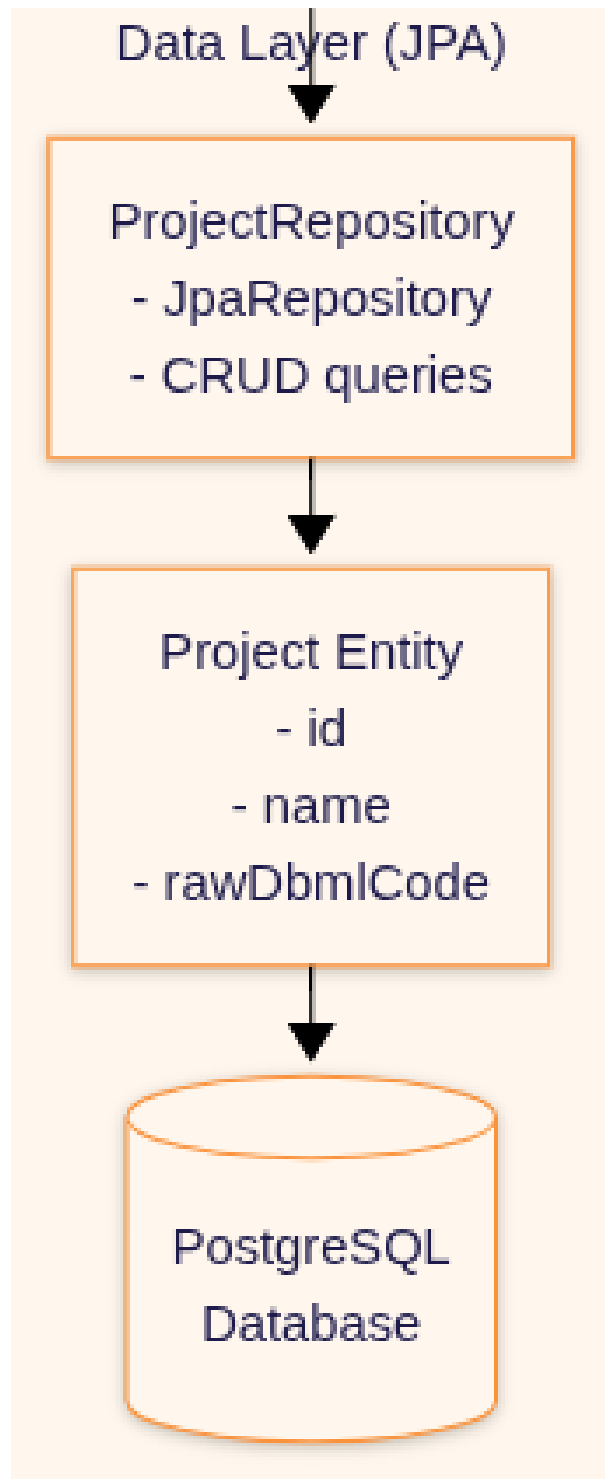
Lớp cơ sở dữ liệu cung cấp nền tảng lưu trữ cho toàn hệ thống DBArchitect và được triển khai bằng PostgreSQL nằm trong một Docker Container. Nó lưu trữ trạng thái dự án được tạo ra trong quy trình làm việc hội thoại, đáng chú ý nhất là tên dự án và biểu diễn DBML liên quan. Cấu hình kết nối được định nghĩa trong `application.properties`, nơi ứng dụng được kết nối với một phiên bản PostgreSQL cục bộ và được cấu hình để sử dụng chế độ cập nhật lược đồ của Hibernate¹. Điều này làm cho việc lưu trữ trở nên minh bạch đối với phần còn lại của hệ thống trong quá trình phát triển, đồng thời vẫn cung cấp cho các dịch vụ Backend một kho lưu trữ quan hệ ổn định.

Về mặt kiến trúc, lớp này rất quan trọng vì nó giúp cho quy trình làm việc của hệ thống bền vững. Dịch vụ Python Agent có thể tạo ra DBML và Java Backend có thể tạo ra mã, nhưng cơ sở dữ liệu mới là thứ giúp kết quả có thể tái sử dụng giữa các phiên: các dự án có thể được liệt kê, tải lại, chỉnh sửa và xuất ra mã nguồn sau này. Nói cách khác, lớp cơ sở dữ liệu biến hệ thống DBArchitect từ một trình tạo tạm thời thành một nền tảng mô hình hóa có trạng thái. Nó hỗ trợ tính liên tục của dự án, chỉnh sửa theo phiên bản và tạo ra các thành phần tiếp theo bằng cách bảo toàn DBML chuẩn cho mỗi dự án.

3.2.5 Luồng dữ liệu của hệ thống

Quy trình xử lý dữ liệu bắt đầu khi người dùng nhập vào yêu cầu thông qua giao diện người dùng React. Giao diện người dùng thu nhận tên dự án và yêu cầu bằng

¹Chế độ cập nhật là chế độ chỉ có cập nhật khi cấu trúc của cơ sở dữ liệu thay đổi, nếu không có thì giữ nguyên cấu trúc từ trước đó. Đây là một cài đặt trong file `applications.properties` thuộc một ứng dụng Spring Boot.



Hình 3.5: Sơ đồ kiến trúc cơ sở dữ liệu

ngôn ngữ tự nhiên , đóng gói chúng thành một yêu cầu và chuyển tiếp đến dịch vụ Python Agent; trạng thái giao diện người dùng (lịch sử trò chuyện, nội dung trình chỉnh sửa) được duy trì cục bộ để bảo toàn ngữ cảnh và cho phép phản hồi ngay lập tức giúp tăng trải nghiệm người dùng. Nếu tác nhân Product Manager xác định đầu vào không rõ ràng, nó sẽ trả về các câu hỏi làm rõ có cấu trúc mà giao diện người dùng hiển thị; câu trả lời của người dùng được gửi lại cho dịch vụ Python Agent, tạo thành một vòng lặp lặp đi lặp lại có kiểm soát để tinh chỉnh đặc tả yêu cầu cho đến khi tác nhân Product Manager xác định là yêu cầu đã rõ ràng.

Sau khi các yêu cầu được xác nhận, tác nhân Architect sẽ chuyển đổi đặc tả đã được làm rõ dưới dạng JSON thành dạng biểu diễn DBML chuẩn và thực hiện kiểm tra cú pháp/ngữ nghĩa. Code DBML đã được làm sạch là đầu vào cho quy trình chuyển đổi của Java Backend: MainService và CodeGenerator chuyển đổi code DBML thành các thành mã nguồn Java Spring Boot.

Cuối cùng, Java Backend lưu trữ bản ghi dự án (bao gồm cả DBML thô) trong PostgreSQL và trả về mã định danh dự án cho giao diện người dùng, cho phép truy xuất và chỉnh sửa sau này. Giao diện người dùng có thể yêu cầu xem trước mã nguồn dưới dạng cây, hoặc tệp ZIP được đóng gói để tải xuống; mỗi hành động tiếp theo này đều sử dụng code DBML đã được làm sạch để đảm bảo đầu ra ổn định.

3.3 Các biểu đồ mô tả chi tiết thiết kế hệ thống

3.4 Thiết kế cơ sở dữ liệu

3.4.1 Mô tả thực thể dữ liệu

Cơ sở dữ liệu của hệ thống được xây dựng xoay quanh thực thể chính: Project (Dự án): Đây là thành phần quan trọng nhất, lưu trữ toàn bộ dữ liệu và trạng thái của một phiên thiết kế cơ sở dữ liệu. Mỗi bản ghi trong thực thể này đại diện cho một không gian làm việc riêng biệt, nơi lưu giữ yêu cầu từ người dùng và kết quả xử lý từ các tác nhân AI.

name: Tên định danh của dự án để người dùng dễ dàng quản lý.

rawDbmlCode: Lưu trữ mã nguồn lược đồ dưới định dạng DBML được sinh ra bởi tác nhân Architect.

status: Trạng thái hiện tại của dự án (ví dụ: Drafting, Validated, Error), giúp hệ thống xác định bước xử lý tiếp theo trong luồng công việc.

createdAt & updatedAt: Các mốc thời gian tự động được hệ thống ghi lại để theo dõi quá trình phát triển của dự án.

3.4.2 Từ điển dữ liệu

Dưới đây là đặc tả chi tiết các trường thông tin trong bảng dữ liệu project, tương ứng với cấu trúc định nghĩa trong mã nguồn hệ thống:

Bảng 3.1: Đặc tả chi tiết thực thể Project

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AI	Mã định danh duy nhất của dự án.
name	VARCHAR(255)	Not Null	Tên gọi của dự án thiết kế.
raw_dbml_code	TEXT	-	Mã nguồn DBML do AI sinh ra hoặc người dùng chỉnh sửa.
status	VARCHAR(50)	-	Trạng thái vận hành hiện tại của dự án.
created_at	TIMESTAMP	Not Null	Thời điểm khởi tạo dự án (Tự động sinh).
updated_at	TIMESTAMP	Not Null	Thời điểm cập nhật cuối cùng (Tự động sinh).

(Ghi chú: PK: Primary Key, AI: Auto Increment)

Chương 4

Thực nghiệm và đánh giá

4.1 Môi trường triển khai và cài đặt hệ thống

4.1.1 Ngôn ngữ lập trình và Framework sử dụng

Hệ thống DBArchitect được triển khai dưới dạng ứng dụng đa tầng với ba lớp chính giao tiếp với nhau thông qua API REST. Ngôn ngữ lập trình và Framework sử dụng của các thành phần thuộc ba lớp này được mô tả trong Bảng 4.1

Bảng 4.1: Tổng hợp đặc tả công nghệ và vai trò các thành phần trong hệ thống

Thành phần	Công nghệ	Thực thi	Mục đích và Vai trò
Frontend	React 18.3.1 (TS)	Node.js	Giao diện người dùng, quản lý trạng thái thiết kế và render sơ đồ ERD trực quan.
Visualization	ReactFlow, Dagre	Node.js	Tương tác kéo thả, tự động bố cục các thực thể và quan hệ trong sơ đồ.
Backend	Spring Boot 4.0.0	Java 21	Điều phối API, quản lý dữ liệu bền vững và thực hiện logic sinh mã nguồn tự động.
Agent System	FastAPI, LangGraph	Python 3.9+	Quản lý quy trình đa tác nhân dựa trên máy trạng thái và điều hướng suy luận AI.
Lỗi LLM	Groq LLaMA	-	Lỗi suy luận ngôn ngữ tự nhiên, trích xuất thực thể và hiểu ngữ nghĩa yêu cầu.
Database	PostgreSQL 14+	Postgres	Lưu trữ tập trung các dự án, lược đồ thiết kế và nhật ký tương tác của hệ thống.

4.1.2 Cài đặt và triển khai hệ thống

4.1.3 Yêu cầu tiên quyết

Hệ thống yêu cầu các môi trường ngôn ngữ cũng như các công cụ quản lý dự án như sau:

- Java 21 (JDK 21)
- Maven (hoặc dùng ./mvnw là lớp wrapper của Maven)
- Node.js 16+ và npm
- Python 3.9+ và pip (có thể sử dụng môi trường ảo Python)
- PostgreSQL (local hoặc container)

4.1.4 Cấu hình cơ sở dữ liệu

Tạo database và user:

```
sudo -u postgres psql
```

```
-c "CREATE ROLE admin WITH LOGIN PASSWORD 'admin123';"
```

```
sudo -u postgres psql -c "CREATE DATABASE dbarchitect OWNER admin;"
```

Nếu sử dụng Docker:

```
docker run -d -name dbarchitect-postgres -e POSTGRES_USER=admin
```

```
-e POSTGRES_PASSWORD=admin123 -e POSTGRES_DB=dbarchitect
```

```
-p 5432:5432 postgres:14
```

4.1.5 Chạy Python Agent Service

Cài dependency:

```
cd agents
```

```
python -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

Chạy server dev:

```
uvicorn api:app -host 0.0.0.0 -port 8000
```

4.1.6 Chạy Java Backend

Build và chạy:

```
cd backend
```

```
./mvnw clean package
```

```
./mvnw spring-boot:run
```

```
java -jar target/backend-0.0.1-SNAPSHOT.jar // file jar đã được đóng gói
```

4.1.7 Chạy Frontend

Cài các thư viện và chạy:

```
cd frontend
```

```
npm install
```

```
npm start
```

4.2 Kịch bản kiểm thử

4.3 Kết quả kiểm thử

4.4 Phân tích và thảo luận kết quả

Kết luận

Tóm tắt bài toán, giải pháp và các kết quả đạt được

Tài liệu tham khảo